

El código de Golay extendido (24, 12)

Trabajo Práctico 2

En grupos de dos personas, se recorre un mismo código en tres niveles: el cálculo de sus propiedades, un modelo de referencia en software y una implementación en RTL. Los resultados numéricos de las tres partes deben coincidir. El trabajo experimental se hace en el dominio de los patrones de error, con $\mathbf{r} = \mathbf{v} + \mathbf{e}$ y $\mathbf{e}$ elegido explícitamente: no se simula ningún canal. El código está generado por la matriz $\mathbf{B}$ que sigue, idéntica para todos los grupos, simétrica y con $\mathbf{B}^2 = \mathbf{I}$, de modo que $\mathbf{G} = [\mathbf{I}_{12} \mid \mathbf{B}]$ y $\mathbf{H} = [\mathbf{B} \mid \mathbf{I}_{12}]$.

Fila	Hex	Binario	Fila	Hex	Binario
$\mathbf{b}_0$	98F	100110001111	$\mathbf{b}_6$	53D	010100111101
$\mathbf{b}_1$	4E7	010011100111	$\mathbf{b}_7$	2BE	001010111110
$\mathbf{b}_2$	357	001101010111	$\mathbf{b}_8$	87B	100001111011
$\mathbf{b}_3$	BE2	101111100010	$\mathbf{b}_9$	E74	111001110100
$\mathbf{b}_4$	DD1	110111010001	$\mathbf{b}_{10}$	F1A	111100011010
$\mathbf{b}_5$	7CC	011111001100	$\mathbf{b}_{11}$	EA9	111010101001

Convención de bits. $\text{cw}[23:12]$ = mensaje y $\text{cw}[11:0]$ = paridad. El bit $\text{m}[11-i]$ es m_i , y el bit $[11-j]$ de $\mathbf{b}_i$ es $B[i][j]$. Rige en las tres partes y en los vectores de prueba.

Parte A — Propiedades del código

Ejercicio 1 Matrices y parámetros

Escribir $\mathbf{G}$ y $\mathbf{H}$ a partir de $\mathbf{B}$ y verificar que $\mathbf{G}\mathbf{H}^T = \mathbf{0}$ y $\mathbf{B}^2 = \mathbf{I}$. Indicar dimensión, tasa y *overhead*. Codificar a mano $\mathbf{m} = 0xA5C$ y comprobar que la palabra obtenida cumple $\mathbf{v}\mathbf{H}^T = \mathbf{0}$.

Ejercicio 2 Distancia mínima y capacidad de corrección

Determinar la distancia mínima, la capacidad de corrección t y la capacidad de detección a partir de la distribución de pesos del código:

Peso	0	8	12	16	24
Cantidad de palabras	1	759	2576	759	1

Ejercicio 3 Decodificación manual

Decodificar a mano las siguientes palabras, siguiendo el algoritmo de cuatro casos de la Parte C. Indicar en cada caso el síndrome, el caso en el que se resuelve, el patrón de error y el mensaje decodificado.

$\mathbf{r}_1$	$\mathbf{r}_2$	$\mathbf{r}_3$
0xA5D9A6	0xA5F9A4	0xA5C9AA

Parte B — Modelo de referencia en software

Se parte de las clases desarrolladas en el Trabajo Práctico 1. El modelo resultante es el *golden model* contra el cual se verifica el RTL.

Ejercicio 4 De $GF(2^m)$ a $GF(2)$

Instanciar la clase de campo de Galois del Trabajo Práctico 1 con $m = 1$ y $P(x) = x + 1$, de modo que sus operaciones se reduzcan a las de $GF(2)$, y verificar las tablas de suma, producto e inverso. Si alguna parte de la implementación asume $m \geq 2$, corregirla y documentar el ajuste. Sobre esa clase, agregar el álgebra matricial que el trabajo anterior no cubría: producto vector–matriz, matriz–matriz y peso de Hamming.

Ejercicio 5 Construcción y verificación

Construir **G** y **H** con las operaciones del punto anterior y verificar las propiedades del Ejercicio 1. Enumerar las 4096 palabras código, obtener la distribución de pesos por fuerza bruta y compararla con la del Ejercicio 2.

Ejercicio 6 Codificador y decodificador de referencia

Implementar el codificador matricial y el decodificador, con los cuatro casos del algoritmo y la rama de no corregible. La tabla de síndromes debe generarse de forma programática a partir de **H**. El decodificador devuelve el mensaje, el patrón de error y las banderas de corrección y de no corregible.

Ejercicio 7 Caracterización del decodificador

- a) Recorrer, para cada peso w entre 0 y 4, la totalidad de los patrones de error de ese peso, clasificando el resultado en tres categorías: corregido de forma exacta, decodificado a otra palabra código, y detectado como no corregible. Los valores deben ser exactamente estos:

Peso de e	0	1	2	3	4
Patrones	1	24	276	2024	10626
Corregidos	1	24	276	2024	0
Detectados	0	0	0	0	10626

- b) Generar los vectores de prueba para la Parte C: un conjunto por submódulo y uno para el decodificador completo, cubriendo las cinco ramas del algoritmo. Documentar el formato.

Parte C — Implementación RTL

La implementación es totalmente pipelineada, con latencia fija y una palabra por ciclo. Todas las entradas llevan el prefijo **i_** y todas las salidas el prefijo **o_**; el reset **i_rst** es sincrónico y activo alto. Las interfaces son obligatorias: los nombres de módulo y de puerto deben respetarse tal cual, porque los bancos de prueba ya armados se conectan directamente a ellos.

```

module golay_mult_b (
    input wire [11:0] i_vec,
    output wire [11:0] o_vec
);

module popcount12 (
    input wire [11:0] i_vec,
    output wire [3:0] o_weight
);

module golay_encoder (
    input wire i_clk, i_rst,
    input wire [11:0] i_msg,
    output reg [23:0] o_cw
);

module golay_decoder (
    input wire i_clk, i_rst,
    input wire [23:0] i_rx,
    output reg [11:0] o_msg,
    output reg [23:0] o_err,
    output reg o_corrected,
    output reg o_uncorrectable
);

```

```

module golay_syndrome (
    input wire [23:0] i_rx,
    output wire [11:0] o_syn
);

module golay_row_search (
    input wire [11:0] i_vec,
    output wire o_found,
    output wire [3:0] o_idx,
    output wire [11:0] o_res
);

module golay_correct (
    input wire [23:0] i_rx, i_err,
    output wire [23:0] o_cw,
    output wire [11:0] o_msg,
    output wire o_corrected
);

module golay_err_gen (
    input wire [11:0] i_syn, i_q,
    output wire [11:0] i_res_syn, i_res_q,
    input wire [3:0] i_w_syn, i_w_q,
    output wire [11:0] i_idx_syn, i_idx_q,
    input wire i_found_syn, i_found_q,
    output wire [23:0] o_err,
    output wire o_uncorrectable
);

```

Ejercicio 8 Codificador

`golay_mult_b` multiplica un vector de 12 bits por **B** y es el único lugar del diseño donde la matriz aparece; junto con `popcount12`, se instancia en el codificador y en el decodificador. El codificador es una instancia de `golay_mult_b`, la concatenación del mensaje con la paridad y un registro de salida, con latencia de un ciclo.

Ejercicio 9 Decodificador

Los cuatro submódulos son combinacionales. `golay_syndrome` calcula $\mathbf{s} = \mathbf{r}[23:12] \cdot \mathbf{B} \oplus \mathbf{r}[11:0]$. `golay_row_search` evalúa en paralelo los doce candidatos $\mathbf{v} \oplus \mathbf{b}_i$, indica si alguno tiene peso menor o igual a 2 y resuelve empates por prioridad de índice; se instancia dos veces. `golay_err_gen` implementa la prioridad de los cuatro casos, donde $\mathbf{u}_i$ es el vector de 12 bits con un uno en la posición i .

Caso	Condición	Patrón de error
1	$w(\mathbf{s}) \leq 3$	$\mathbf{e} = (\mathbf{0} \mid \mathbf{s})$
2	$\exists i : w(\mathbf{s} \oplus \mathbf{b}_i) \leq 2$	$\mathbf{e} = (\mathbf{u}_i \mid \mathbf{s} \oplus \mathbf{b}_i)$
3	$w(\mathbf{q}) \leq 3$, con $\mathbf{q} = \mathbf{s} \cdot \mathbf{B}$	$\mathbf{e} = (\mathbf{q} \mid \mathbf{0})$
4	$\exists i : w(\mathbf{q} \oplus \mathbf{b}_i) \leq 2$	$\mathbf{e} = (\mathbf{q} \oplus \mathbf{b}_i \mid \mathbf{u}_i)$
5	ninguna de las anteriores	<code>o_uncorrectable</code>

El decodificador integra los submódulos en un pipeline de tres etapas, con latencia fija de 3 ciclos y una palabra por ciclo. Si `o_uncorrectable` está en alto, `o_msg` y `o_err` no son válidas.

Etapas	Lógica combinacional	Registros de salida
E1	<code>golay_syndrome</code>	$\mathbf{s}, \mathbf{r}$
E2	<code>popcount12(s), golay_row_search(s), golay_mult_b(s) → q, popcount12(q), golay_row_search(q)</code>	$w(\mathbf{s}), w(\mathbf{q}), \mathbf{q}$, ambas búsquedas, $\mathbf{r}$
E3	<code>golay_err_gen, golay_correct</code>	salidas del decodificador

Ejercicio 10 Verificación y análisis

Un banco de prueba por submódulo y uno para el decodificador completo, alimentados con los vectores del Ejercicio 7. Como mínimo: `golay_mult_b` aplicado dos veces sobre los 4096 vectores debe recuperar la entrada, que es la comprobación en hardware de $\mathbf{B}^2 = \mathbf{I}$; barrido exhaustivo de `popcount12` y `golay_row_search`; síndrome nulo para las 4096 palabras código; y el decodificador completo verificando el mensaje y el patrón de error bit a bit. Reportar latencia y cantidad de instancias de cada submódulo.

Parte D — Punto bonus

Ejercicio 11 Interleaver convolucional

Implementar en RTL un *interleaver* convolucional que opera bit a bit, con λ ramas e incremento de retardo J , de modo que la rama i retarda $i \cdot J$ bits. Tras el reset el conmutador arranca en la rama 0. El *deinterleaver* es su espejo: la rama i retarda $(\lambda - 1 - i) \cdot J$ bits.

```
module conv_interleaver #(
    parameter LAMBDA = 24,
    parameter J      = 1
) (
    input  wire i_clk,
    input  wire i_rst,
    input  wire i_bit,
    output reg o_bit
);
```

```
module conv_deinterleaver #(
    parameter LAMBDA = 24,
    parameter J      = 1
) (
    input  wire i_clk,
    input  wire i_rst,
    input  wire i_bit,
    output reg o_bit
);
```

Entrega

Se entrega un informe en PDF con los cálculos de la Parte A, los resultados de la Parte B y el análisis de la Parte C, junto con el modelo de referencia, el código RTL con cada submódulo en un archivo separado, y los bancos de prueba y vectores. Ambos integrantes deben estar en condiciones de explicar cualquier parte del trabajo.

El informe debe responder además por qué el síndrome no depende del mensaje transmitido sino solo del patrón de error, y por qué una distancia mínima de 8 permite detectar cuatro errores pero no corregirlos.